

Learn SQL in 10 Parts + 15 SQL Tips & Tricks (Free Course Bonus) — Free Practice Guide

Beyond School · Ready to learn in a fun way! Anyone can learn — easy! Nothing to install, just a web browser.

Step 1: Open a free online SQL playground

- sqliteonline.com — choose "SQLite", paste, click Run.
- db-fiddle.com — pick "SQLite" as the database, paste and Run.
- sqlime.org — a simple SQLite editor in your browser.
- sqlbolt.com — free interactive lessons for extra practice.

Step 2: Paste this setup script and click Run

It creates the same library tables used in the video. If anything goes wrong, just run it again in a fresh playground.

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  title TEXT,
  author TEXT,
  year INTEGER,
  genre TEXT,
  pages INTEGER
);
INSERT INTO books VALUES
  (1, 'Harry Potter', 'J.K. Rowling', 1997, 'Fantasy', 309),
  (2, 'Matilda', 'Roald Dahl', 1988, 'Comedy', 240),
  (3, 'Wonder', 'R.J. Palacio', 2012, 'Drama', 310),
  (4, 'The Hobbit', 'J.R.R. Tolkien', 1937, 'Fantasy', 310),
  (5, 'Holes', 'Louis Sachar', 1998, 'Mystery', 233),
  (6, 'Charlotte''s Web', 'E.B. White', 1952, 'Classic', 192),
  (7, 'Dog Man', 'Dav Pilkey', 2016, 'Comedy', 240),
  (8, 'Coraline', 'Neil Gaiman', 2002, 'Fantasy', 162);

CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  name TEXT,
  city TEXT,
  joined TEXT,
```

```
  email TEXT
);
INSERT INTO members VALUES
  (1, 'Aisha', 'Austin', '2023-01-15', 'aisha@mail.com'),
  (2, 'Ben', 'Denver', '2023-03-02', NULL),
  (3, 'Carlos', 'Austin', '2024-06-20', 'carlos@mail.com'),
  (4, 'Dana', 'Boston', '2024-09-10', NULL),
  (5, 'Eli', 'Denver', '2025-02-01', 'eli@mail.com');

CREATE TABLE loans (
  loan_id INTEGER PRIMARY KEY,
  book_id INTEGER,
  member_id INTEGER,
  loan_date TEXT,
  return_date TEXT
);
INSERT INTO loans VALUES
  (1, 1, 1, '2026-01-05', '2026-01-19'),
  (2, 3, 1, '2026-02-10', NULL),
  (3, 2, 2, '2026-01-20', '2026-02-03'),
  (4, 4, 3, '2026-03-01', '2026-03-15'),
  (5, 1, 3, '2026-04-02', NULL),
  (6, 7, 4, '2026-04-10', '2026-04-24'),
  (7, 7, 2, '2026-05-01', NULL),
  (8, 5, 4, '2026-05-15', NULL);
```

Step 3: Run every query from the video, in order

Paste one at a time, click Run, and compare with the expected result.

Tip 1: Aliases

```
SELECT b.title AS book, m.name AS member FROM loans AS l JOIN books AS b ON b.id = l.book_id JOIN members AS m ON m.member_id = l.member_id WHERE l.return_date IS NULL ORDER BY b.title;
```

```
Expected result:
book | member
Dog Man | Ben
Harry Potter | Carlos
Holes | Dana
Wonder | Aisha
```

Tip 2: DISTINCT

```
SELECT DISTINCT genre FROM books ORDER BY genre;
```

```
Expected result:
genre
Classic
Comedy
Drama
Fantasy
Mystery
```

Tip 3: COALESCE

```
SELECT name,   COALESCE(email, 'no email yet') AS email FROM members;
```

```
Expected result:
name | email
Aisha | aisha@mail.com
Ben | no email yet
Carlos | carlos@mail.com
Dana | no email yet
Eli | eli@mail.com
```

Tip 4: CASE counting

```
SELECT   SUM(CASE WHEN pages >= 300 THEN 1 ELSE 0 END) AS long_books,   SUM(CASE WHEN pages < 300 THEN 1 ELSE 0 END) AS short_books FROM books;
```

```
Expected result:
long_books | short_books
3 | 5
```

Tip 5: LIKE wildcards

```
SELECT title FROM books WHERE title LIKE '_o%';
```

```
Expected result:
title
Wonder
Holes
Dog Man
Coraline
```

Tip 6: BETWEEN and IN

```
SELECT title, year FROM books WHERE year BETWEEN 1990 AND 2010;
```

```
Expected result:
title | year
Harry Potter | 1997
Holes | 1998
Coraline | 2002
```

Tip 7: Top N

```
SELECT title, pages FROM books ORDER BY pages DESC LIMIT 3;
```

```
Expected result:
title | pages
Wonder | 310
The Hobbit | 310
Harry Potter | 309
```

Trap 1: = NULL

```
SELECT name FROM members WHERE email = NULL;
```

```
Expected result:
name
```

Fix: IS NULL

```
SELECT name FROM members WHERE email IS NULL;
```

```
Expected result:
name
Ben
Dana
```

Trap 2: GROUP BY columns

```
SELECT genre, title, COUNT(*) AS books FROM books GROUP BY genre;
```

```
Expected result:
genre | title | books
Classic | Charlotte's Web | 1
Comedy | Matilda | 2
Drama | Wonder | 1
Fantasy | Harry Potter | 3
Mystery | Holes | 1
```

Trap 3: WHERE vs HAVING

```
SELECT genre, COUNT(*) AS books FROM books WHERE year > 1950 GROUP BY genre HAVING COUNT(*) >= 2;
```

```
Expected result:
genre | books
Comedy | 2
Fantasy | 2
```

Tip 12: SELECT before UPDATE

```
SELECT * FROM members WHERE member_id = 5;
```

```
Expected result:
member_id | name | city | joined | email
5 | Eli | Denver | 2025-02-01 | eli@mail.com
```

Then UPDATE

```
UPDATE members SET city = 'Dallas' WHERE member_id = 5;
SELECT member_id, name, city FROM members;
```

```
Expected result:
member_id | name | city
1 | Aisha | Austin
2 | Ben | Denver
3 | Carlos | Austin
4 | Dana | Boston
5 | Eli | Dallas
```

Tip 13: EXPLAIN

```
EXPLAIN QUERY PLAN SELECT * FROM books WHERE author = 'Roald Dahl';
EXPLAIN QUERY PLAN SELECT * FROM books WHERE author = 'Roald Dahl';
```

```
Expected result:
id | parent | notused | detail
2 | 0 | 0 | SCAN books
```

```
Expected result:
id | parent | notused | detail
2 | 0 | 0 | SCAN books
```

Tip 14: add an index

```
CREATE INDEX idx_author ON books(author);
EXPLAIN QUERY PLAN SELECT * FROM books WHERE author = 'Roald Dahl';
```

```
Expected result:
id | parent | notused | detail
3 | 0 | 0 | SEARCH books USING INDEX idx_author (author=?)
```

Tip 15: CTEs for readability

```
WITH still_out AS (    SELECT member_id, COUNT(*) AS n    FROM loans WHERE return_date IS NULL    GROUP BY member_id) SELECT m.name, s.n AS books_out FROM still_out AS s JOIN members AS m ON m.member_id = s.member_id;
```

```
Expected result:
name | books_out
Aisha | 1
Ben | 1
Carlos | 1
Dana | 1
```

Step 4: Challenges (try first!)

C1. Show the 2 newest books with a CASE label: 'Classic era' before 1990, otherwise 'Modern'.

C2. Count Fantasy books and non-Fantasy books in ONE row using SUM(CASE ...).

C3. List loans that are still out (no return date) and started before May 2026. Use IS NULL, not = NULL!

Answers

C1. Show the 2 newest books with a CASE label: 'Classic era' before 1990, otherwise 'Modern'.

```
SELECT title, year,
  CASE WHEN year < 1990 THEN 'Classic era' ELSE 'Modern' END AS era
FROM books ORDER BY year DESC LIMIT 2;
```

```
Expected result:
title | year | era
Dog Man | 2016 | Modern
Wonder | 2012 | Modern
```

C2. Count Fantasy books and non-Fantasy books in ONE row using SUM(CASE ...).

```
SELECT SUM(CASE WHEN genre = 'Fantasy' THEN 1 ELSE 0 END) AS fantasy,
  SUM(CASE WHEN genre != 'Fantasy' THEN 1 ELSE 0 END) AS other
FROM books;
```

```
Expected result:
fantasy | other
3 | 5
```

C3. List loans that are still out (no return date) and started before May 2026. Use IS NULL, not = NULL!

```
SELECT * FROM loans
WHERE return_date IS NULL AND loan_date < '2026-05-01';
```

```
Expected result:
loan_id | book_id | member_id | loan_date | return_date
2 | 3 | 1 | 2026-02-10 | NULL
5 | 1 | 3 | 2026-04-02 | NULL
```

Keep going!

Watch the whole free SQL course on Beyond School, starting with SQL Part 1: https://youtu.be/72JvYu2OtiA
Beyond School — Ready to learn in a fun way!